



Universidade Federal de Uberlândia

Faculdade de Engenharia Elétrica

Professores Alan Petrônio, Marcelo Rodrigues, Kil Jin e Renato Carrijo

Versão 1.0 – 2026/1 | Data da atividade: 19/06/2026

Roteiro-tutorial: Atividades Acadêmicas

TDE – Trabalho Discente Efetivo

Duração: 4 horários de 50 minutos

Curso:	Bacharelado em Cibersegurança (noturno) – FEELT
Módulo:	2 – Aplicação para ambiente WEB (1º período)
Tema:	Back-end Flask, integração cliente–servidor e autenticação
Mecanismo:	Cada estudante publica o código no seu próprio repositório GitHub
Pré-requisito:	Lógica de programação (Módulo 1); Python instalado

1 Visão geral e alinhamento com o Módulo 2

Neste roteiro você vai construir, do zero, uma pequena API em **Flask** (o servidor) e um **script Python** que consome essa API (o cliente). A ideia central é simples e poderosa: *primeiro você cria o lado servidor que disponibiliza um dado; logo em seguida você escreve o cliente que consome esse dado*. Repetiremos esse ciclo quatro vezes, sempre partindo de um exemplo simples e evoluindo para um mais completo:

1. **Horário 1:** servidor Flask + rota **GET** `/api/soma` (exemplo mais simples possível).
2. **Horário 2:** cliente Python com a biblioteca **requests** consumindo a rota via **GET**.
3. **Horário 3:** método **POST**, dados no **corpo** da requisição e parâmetros no **cabeçalho** (*header*).
4. **Horário 4:** rota protegida com **Bearer token** no cabeçalho **Authorization**.

Esse recorte está diretamente ancorado na ementa do Módulo 2. O conteúdo programático 3 (*Back-end, Dados e Integração*) cita literalmente “camada de aplicação; servidores de aplicação; **APIs REST; integração cliente–servidor**”. Já os objetivos de aprendizagem do módulo pedem “planejar e executar projetos ágeis, conduzindo **uso de Git**” (objetivo 2), “implementar back-end com camadas de aplicação” (objetivo 4), “compreender a arquitetura da Web e os protocolos (HTTP/HTTPS)” (objetivo 6) e “aplicar princípios de segurança e privacidade no ciclo de vida do software” (objetivo 7). O Horário 4, com autenticação por token, encosta justamente nesse último ponto.

Uma observação importante: a rota `api/soma` usa um *verbo* (“soma”) na URL, o que é ótimo para aprender o mecanismo, mas não é REST “puro” – REST modela *recursos* (substantivos), não ações.

Trate `api/soma` como uma simplificação didática; em módulos futuros você modelará recursos de verdade (ex.: `/usuarios`, `/produtos`).

2 Setup inicial (fazer uma única vez, ≈10 min)

1. Verifique o Python (3.8 ou superior):

```
python --version      # se não funcionar, tente: python3 --version
```

2. Crie a pasta do projeto e o ambiente virtual:

```
mkdir api-soma
cd api-soma
python -m venv venv
# Windows:      venv\Scripts\activate
# Linux / macOS: source venv/bin/activate
```

3. Instale as bibliotecas:

```
pip install flask requests
```

4. Configure o Git (uma vez na vida, na sua máquina):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

5. Crie um repositório vazio no GitHub: entre em <https://github.com/new>, dê o nome `api-soma`, deixe-o **público**, e *não* marque a opção de criar README (faremos isso pela máquina). Crie também um arquivo `.gitignore` contendo a pasta `venv/`. No Horário 4 usaremos um token: tokens, senhas e chaves nunca devem ser versionados no GitHub.

3 Horário 1 — O primeiro servidor Flask (GET `/api/soma`)

Objetivos do horário

Ao final dos 50 minutos você terá um servidor Flask rodando localmente, respondendo a uma rota `GET` que recebe dois números pela URL e devolve a soma em JSON – e terá feito o primeiro *commit* no seu repositório.

Vídeo sugerido (assista antes de codar)

Assista (~10 min): “Como criar uma API REST do zero com Python e Flask”.

<https://www.youtube.com/watch?v=levz4eumJ98>

Busca alternativa: criar API REST do zero Python Flask.

Arquitetura deste horário

Um único processo: o navegador (cliente) faz uma requisição HTTP; o servidor Flask responde com JSON.

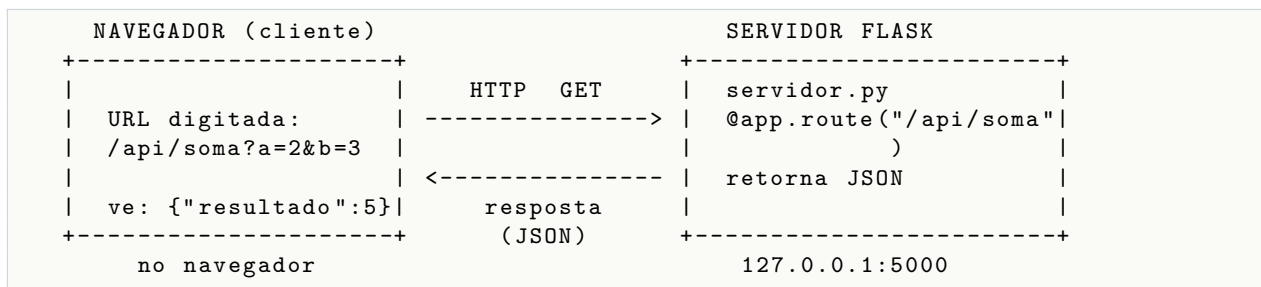


Figura 1: Arquitetura do Horário 1 – navegador e servidor Flask via HTTP GET.

Passo a passo – lado servidor

Crie o arquivo `servidor.py`:

Listing 1: `servidor.py` – versão 1 (GET simples)

```

1 from flask import Flask, request, jsonify
2
3 app = Flask(__name__)
4
5 # Rota 1: GET http://127.0.0.1:5000/api/soma?a=2&b=3
6 @app.route("/api/soma", methods=["GET"])
7 def soma():
8     # Le os parametros 'a' e 'b' da URL (query string)
9     a = request.args.get("a", type=float)
10    b = request.args.get("b", type=float)
11
12    # Validacao simples (passo na direcao de "aplicacoes seguras")
13    if a is None or b is None:
14        return jsonify({"erro": "informe a e b, ex: ?a=2&b=3"}), 400
15
16    return jsonify({"resultado": a + b})
17
18 if __name__ == "__main__":
19     app.run(host="127.0.0.1", port=5000, debug=True)

```

Entenda as rotas no Flask. O decorador `@app.route("/api/soma", methods=["GET"])` diz ao Flask: “quando chegar uma requisição GET para o caminho `/api/soma`, execute a função `soma()`”. O caminho é a **rota**; `methods` define quais verbos HTTP ela aceita.

Rode o servidor:

```
python servidor.py
```

Teste no navegador abrindo:

```
http://127.0.0.1:5000/api/soma?a=2&b=3
```

Você deve ver `{"resultado": 5.0}`. Experimente trocar os números na URL.

Entrega no GitHub

Abra um **segundo terminal** (deixe o servidor rodando no primeiro) e execute:

```
echo "venv/" > .gitignore
```

```
git init
git add .
git commit -m "H1: servidor Flask com GET /api/soma"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/api-soma.git
git push -u origin main
```

Checkpoint – só avance se

(1) O servidor responde no navegador com o JSON correto; (2) seu repositório `api-soma` no GitHub já mostra o arquivo `servidor.py`.

4 Horário 2 — O cliente Python consumindo a API (GET)

Objetivos do horário

Escrever um script Python (`cliente.py`) que, usando a biblioteca `requests`, chama a rota do Horário 1 **localmente** e lê a resposta: código de status, cabeçalhos e corpo em JSON.

Vídeo sugerido (assista antes de codar)

Assista (~12 min): “Api Rest com Python + Flask” (consumo de API).

<https://www.youtube.com/watch?v=LP8besicfH4>

Leitura de apoio (PT): biblioteca Requests – <https://www.digitalocean.com/community/tutorials/how-to-get-started-with-the-requests-library-in-python-pt>

Arquitetura deste horário

Agora são **dois processos Python** conversando por HTTP: o `cliente.py` no lugar do navegador.

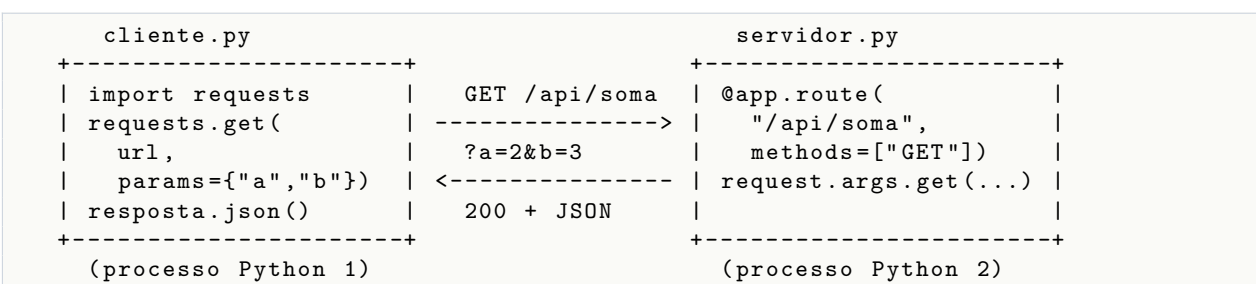


Figura 2: Arquitetura do Horário 2 – dois processos Python conversando por HTTP.

Passo a passo – lado cliente

Mantenha o `servidor.py` rodando no primeiro terminal. Crie o `cliente.py`:

Listing 2: `cliente.py` – consumindo via GET

```
1 import requests
2
3 URL = "http://127.0.0.1:5000/api/soma"
4
5 # Em GET, os parametros vao na "query string": ?a=2&b=3
```

```
6  parametros = {"a": 2, "b": 3}
7
8  resposta = requests.get(URL, params=parametros)
9
10 print("Status:", resposta.status_code)           # 200 = OK
11 print("Cabecalhos:", dict(resposta.headers))      # metadados da resposta
12 print("Corpo (JSON):", resposta.json())           # {'resultado': 5.0}
```

Rode o cliente no segundo terminal:

```
python cliente.py
```

Observe três coisas na saída: o **status** (200 significa sucesso), os **cabecalhos** que o servidor enviou de volta (note o Content-Type: application/json) e o **corpo** já convertido em dicionário Python por `.json()`.

Entrega no GitHub

Versione o novo arquivo:

```
git add cliente.py
git commit -m "H2: cliente Python consumindo a API via GET"
git push
```

Checkpoint – só avance se

O `cliente.py` imprime Status: 200 e o resultado correto; o commit do Horário 2 aparece no GitHub.

5 Horário 3 — POST: dados no corpo e parâmetros no cabeçalho

Objetivos do horário

Adicionar uma rota POST `/api/soma` que lê os números do **corpo** (JSON) e um parâmetro extra do **cabeçalho** (*header*). No cliente, aprender a enviar corpo e cabeçalhos com `requests.post`.

Vídeo sugerido (assista antes de codar)

Assista (~20 min): “Python REST API Tutorial – Building a Flask REST API” (cobre GET e POST).

<https://www.youtube.com/watch?v=GMppyAPbLYk>

Leitura de apoio (PT-BR): cabeçalhos de requisição (MDN) – https://developer.mozilla.org/pt-BR/docs/Glossary/Request_header

GET ou POST? Em GET os dados viajam *visíveis* na URL (`?a=2&b=3`) – bom para *buscar* dados. Em POST os dados vão no **corpo** da requisição, fora da URL – usado para *enviar/criar* dados. Já o **cabeçalho** carrega *metadados* da requisição (quem está chamando, formato do conteúdo, credenciais).

Arquitetura deste horário

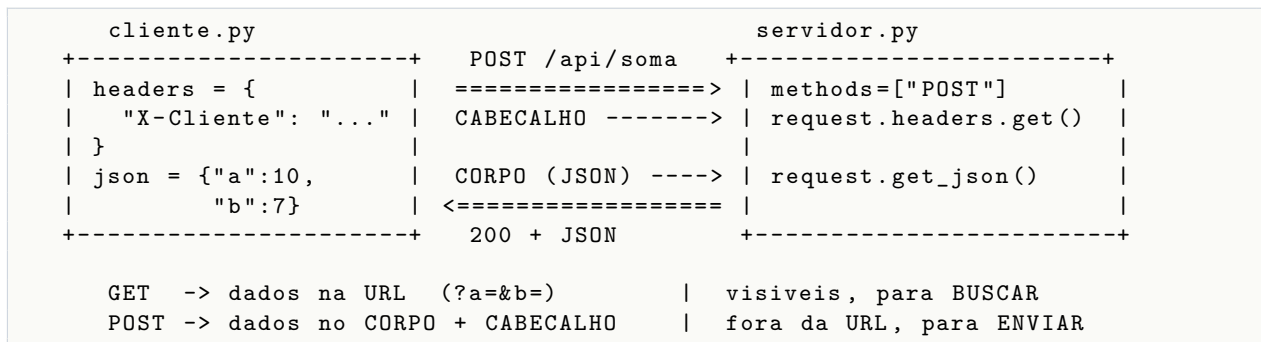


Figura 3: Arquitetura do Horário 3 – POST com dados no corpo e parâmetros no cabeçalho.

Passo a passo – lado servidor

Adicione **esta nova rota** ao `servidor.py` (logo abaixo da função `soma()`). A URL é a mesma, mas agora aceita o verbo POST:

Listing 3: `servidor.py` – nova rota POST (mesmo arquivo)

```
1 # Rota 2: POST http://127.0.0.1:5000/api/soma (dados no CORPO em JSON)
2 @app.route("/api/soma", methods=["POST"])
3 def soma_post():
4     dados = request.get_json(silent=True) or {}
5     a = dados.get("a")
6     b = dados.get("b")
7
8     # Le um parametro enviado no CABECALHO (header) da requisicao
9     cliente = request.headers.get("X-Cliente", "anonimo")
10
11     if a is None or b is None:
12         return jsonify({"erro": "envie a e b no corpo JSON"}), 400
13
14     return jsonify({"resultado": a + b, "chamado_por": cliente})
```

Repare que agora `/api/soma` tem *duas* funções – uma para GET, outra para POST. Isso é normal e desejável: o mesmo endereço responde de formas diferentes conforme o verbo HTTP. Reinicie o servidor (Ctrl+C e `python servidor.py`) para o Flask carregar a nova rota.

Passo a passo – lado cliente

Crie `cliente_post.py`:

Listing 4: `cliente_post.py` – POST com corpo e cabeçalho

```
1 import requests
2
3 URL = "http://127.0.0.1:5000/api/soma"
4
5 # Agora os dados vao no CORPO da requisicao (JSON)
6 corpo = {"a": 10, "b": 7}
7
8 # Parametros que viajam no CABECALHO (header)
```

```
9 cabecalhos = {"X-Cliente": "turma-ciberseguranca"}
10
11 resposta = requests.post(URL, json=corpo, headers=cabecalhos)
12
13 print("Status:", resposta.status_code)
14 print("Corpo (JSON):", resposta.json())
15 # Esperado: {'resultado': 17, 'chamado_por': 'turma-ciberseguranca'}
```

Rode `python cliente_post.py` no segundo terminal. Troque o valor de `X-Cliente` e veja o campo `chamado_por` mudar na resposta: você acabou de *enviar um parâmetro pelo cabeçalho*.

Entrega no GitHub

```
git add servidor.py cliente_post.py
git commit -m "H3: rota POST com corpo JSON e parametro no cabecalho"
git push
```

Checkpoint – só avance se

O `cliente_post.py` recebe `resultado: 17` e o `chamado_por` reflete o cabeçalho enviado; commit do Horário 3 publicado.

6 Horário 4 — Protegendo a rota com Bearer token

Objetivos do horário

Criar uma rota protegida que só responde quando o cliente envia um **Bearer token** no cabeçalho `Authorization`. Testar os três cenários: token válido (200), sem token (401) e token errado (401).

Vídeo sugerido (assista antes de codar)

Assista / leia (~15 min): conceito de Bearer token no cabeçalho `Authorization`.

Busca no YouTube: autenticação API token Bearer Flask.

Referência técnica (a norma): RFC 6750 – <https://datatracker.ietf.org/doc/html/rfc6750>

O que é um Bearer token? É uma credencial em texto que o cliente coloca no cabeçalho `Authorization`, no formato exato:

`Authorization: Bearer <token>`

“*Bearer*” significa “portador”: quem apresenta o token tem acesso, como uma pulseira de evento. O servidor lê esse cabeçalho, confere se o token é válido e, se não for, responde 401 `Unauthorized`.

Arquitetura deste horário

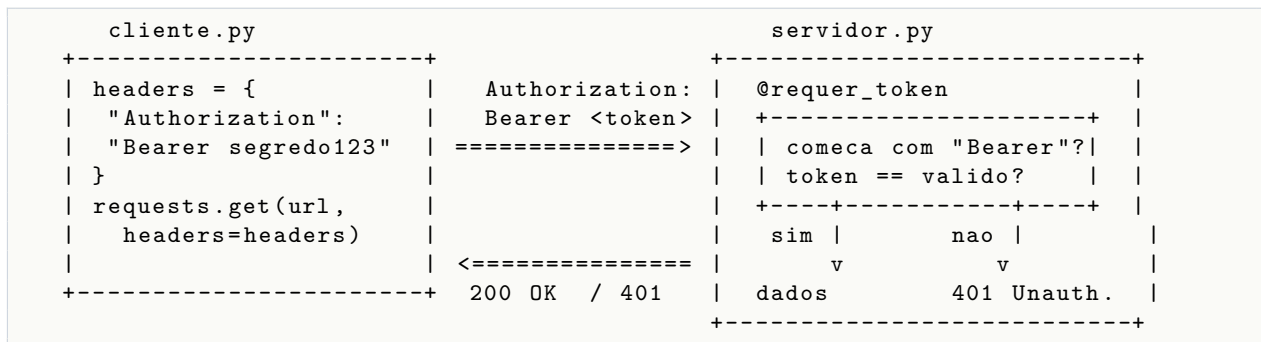


Figura 4: Arquitetura do Horário 4 – rota protegida por Bearer token (200 ou 401).

Passo a passo – lado servidor

No topo do `servidor.py`, importe `wraps`:

```
1 from functools import wraps
```

Adicione o “porteiro” (um *decorador*) e a rota protegida:

Listing 5: `servidor.py` – Bearer token (mesmo arquivo)

```
1 # Em producao, isto viria de variavel de ambiente, NUNCA no codigo!
2 TOKEN_VALIDO = "segredo-da-turma-123"
3
4 def requer_token(funcao):
5     @wraps(funcao)
6     def envoltorio(*args, **kwargs):
7         cabecalho = request.headers.get("Authorization", "")
8
9         # 1) precisa comecar com "Bearer "
10        if not cabecalho.startswith("Bearer "):
11            return jsonify({"erro": "token ausente"}), 401
12
13        # 2) extrai o token depois da palavra "Bearer "
14        token = cabecalho.split(" ", 1)[1]
15
16        # 3) confere se o token e valido
17        if token != TOKEN_VALIDO:
18            return jsonify({"erro": "token invalido"}), 401
19
20        return funcao(*args, **kwargs)
21    return envoltorio
22
23 # Rota protegida: exige Authorization: Bearer <token>
24 @app.route("/api/protegido", methods=["GET"])
25 @requer_token
26 def protegido():
27     return jsonify({"mensagem": "Acesso autorizado! Dados secretos aqui."})
```

Reinicie o servidor.

Passo a passo – lado cliente

Crie `cliente_token.py` e teste os três cenários de uma vez:

Listing 6: `cliente_token.py` – enviando o Bearer token

```
1 import requests
2
3 URL = "http://127.0.0.1:5000/api/protegido"
4 TOKEN = "segredo-da-turma-123"
5
6 # O token vai no cabeçalho, no formato: Bearer <token>
7 cabecalhos = {"Authorization": f"Bearer {TOKEN}"}
8
9 print(">> Com token valido:")
10 r1 = requests.get(URL, headers=cabecalhos)
11 print(" ", r1.status_code, r1.json())
12
13 print(">> Sem token nenhum:")
14 r2 = requests.get(URL)
15 print(" ", r2.status_code, r2.json())
16
17 print(">> Com token errado:")
18 r3 = requests.get(URL, headers={"Authorization": "Bearer chute-errado"})
19 print(" ", r3.status_code, r3.json())
```

Saída esperada: o primeiro caso retorna 200 com a mensagem; os outros dois retornam 401. **Você acabou de implementar controle de acesso por token** – exatamente o tipo de mecanismo que protege APIs reais.

Nota de segurança: em uma aplicação real, o token (1) não deve ficar escrito no código nem ser versionado no GitHub – ele viria de uma variável de ambiente; (2) só é seguro quando trafega sobre **HTTPS**, pois caso contrário qualquer um na rede poderia ler o cabeçalho; e (3) faz parte dos cuidados com dados pessoais exigidos pela LGPD.

Entrega final no GitHub

Confirme que `venv/` está no `.gitignore` e finalize com um bom README.

```
git add servidor.py cliente_token.py
git commit -m "H4: rota protegida com Bearer token (200/401)"
git push
```

Checkpoint – só avance se

Os três cenários imprimem 200, 401 e 401; o repositório final contém os quatro arquivos de cliente/servidor e **nenhum** segredo versionado.

7 Encerramento

Mapa de cobertura – o que cada horário exercitou do Módulo 2

H	O que você fez	Item do Módulo 2
1	Servidor Flask, rota GET, JSON	APIs REST; servidores de aplicação; HTTP
2	Cliente Python com <code>requests</code>	Integração cliente-servidor; arquitetura da Web
3	POST, corpo JSON, cabeçalhos	Métodos HTTP; camada de aplicação
4	Bearer token, 401 Unauthorized	Segurança, controle de acesso, privacidade (LGPD)
Em todos: versionamento e colaboração com Git/GitHub (conteúdo 1.4 + objetivo 2)		

Para aprofundar (próximos passos)

- Trocar o token fixo por **JWT** (token assinado e com validade).
- Ler o token de uma **variável de ambiente** em vez do código.
- Escrever um **teste automatizado** simples para a rota `/api/soma` (objetivo “testáveis”).
- Servir tudo sobre **HTTPS** e entender o que muda na segurança.

Referências

Vídeos (YouTube):

- Como criar uma API REST do zero com Python e Flask. <https://www.youtube.com/watch?v=levz4eumJ98>
- Api Rest com Python + Flask (#019). <https://www.youtube.com/watch?v=LP8besicfH4>
- Python REST API Tutorial – Building a Flask REST API (GET/POST). <https://www.youtube.com/watch?v=GMppyAPbLYk>

Leituras:

- Documentação oficial do Flask. <https://flask.palletsprojects.com/>
- Biblioteca Requests em Python (DigitalOcean, PT). <https://www.digitalocean.com/community/tutorials/how-to-get-started-with-the-requests-library-in-python-pt>
- Cabeçalhos de requisição (MDN, PT-BR). https://developer.mozilla.org/pt-BR/docs/Glossary/Request_header
- RFC 6750 – The OAuth 2.0 Authorization Framework: Bearer Token Usage. <https://datatracker.ietf.org/doc/html/rfc6750>